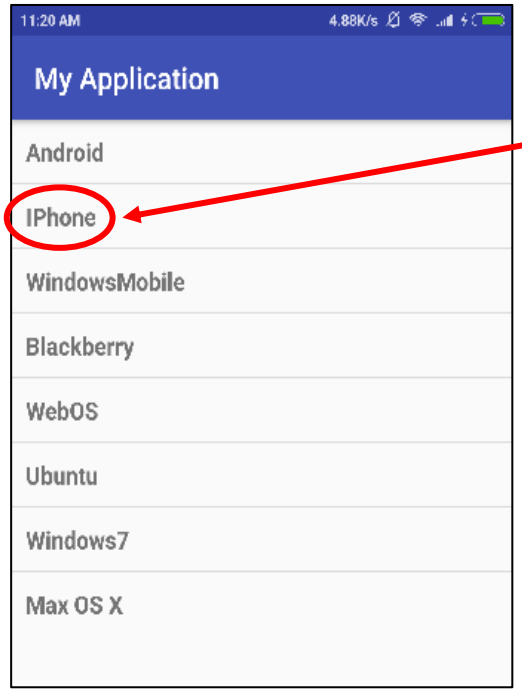
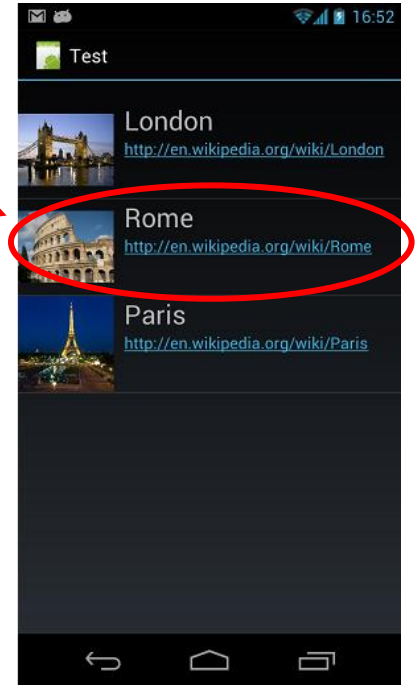
Les composants graphiques complexes

- Introduction
  - Les composants complexes sont des composants de type « groupe » (voir chap.2)
  - Ci-joint une liste non exhaustive des composants de type « groupe »:
    - ListView
    - GridView
    - Gallery
    - Spinner
    - Toolbar
    - ExpandableListView
    - RecyclerView
  - Dans ce chapitre, on va présenter les composants suivants : « *ListView* », « *GridView* », « *Spinner* » et « *RecyclerView* »

- ListView
  - C'est un composant qui affiche des éléments en liste verticale (un simple exemple de ***ListView*** est le carnet de contacts, où vous avez une liste de vos contacts affichés dans une ***ListView***)
  - Les éléments qui constituent une ***ListView*** s'appellent ***ListItem***



ListItem

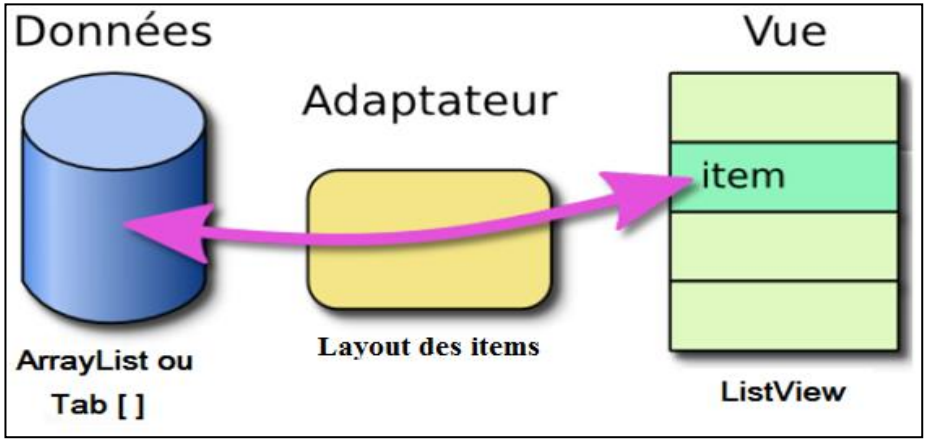


- ***ListItem*** peut contenir un composant simple (numérique, texte, image…) (*ListItem* simple) ou une combinaison de plusieurs composants (*ListItem* composé)

***ListView* avec *ListItems* simples**

***ListView* avec *ListItems* composés**

- ListView
  - Les éléments d'une ***ListView*** admettent un ***layout*** (gestionnaire de disposition) qui définit la disposition et l'apparence des données de la liste
  - Ce ***layout*** est géré par un adaptateur (***adapter***) qui joue le rôle d'intermédiaire entre le composant graphique (*ListView* dans notre cas) et ses données. Il permet de faciliter la gestion des données constituant les éléments (*ListItem*) du composant (*ListView* dans notre cas)
  - L'adaptateur est associé à la liste en utilisant la méthode « ***setAdapter*** » comme suit :



List_name.setAdapter (adapter_name)

- ListView
  - Les données de la liste sont fournies à l'adaptateur en provenance de plusieurs sources possibles (tableau (voir Diapo.7), collection simple, liste de Map, base de données…)
  - L'adaptateur se charge de créer une vue pour chaque élément (*ItemView*) avec un *layout* (*ItemLayout*) à respecter. Puis, les vues sont fournies à la liste pour être affichées dans l'ordre et la forme définie
  - Il existe des adaptateurs prédéfinis **(*ArrayAdapter* , *SimpleAdapter*, *CursorAdapter*)** et vous pouvez aussi construire votre adaptateur personnalisé en créant une classe héritant la classe « ***BaseAdapter*** »
  - Les adaptateurs prédéfinis :
    - ***ArrayAdapter*** : Il permet de gérer les sources de données de types tableau et collection simple
    - ***SimpleAdapter*** : Il est plus flexible que *l'ArrayAdapter* et il permet de gérer les données provenant d'une liste de *Map*
    - ***CursorAdapter*** : Il permet de gérer les données venant d'une base de données

- ListView
  - Une ***ListView*** avec des éléments simples (valeurs numériques, chaines) peut être construite de deux façons :
    - Avec XML, en utilisant un tableau de chaînes (*string-array*) et l'attribut « *entries* » de la liste, défini dans le fichier de *layout* XML

```
<ListView
    android:id="@+id/listView"
    android:layout_width="344dp"
    android:layout_height="495dp"
    android:divider="@android:color/black"
    android:dividerHeight="1dp"
    android:entries="@array/day_array"
    tools:layout_editor_absoluteX="8dp"
    tools:layout_editor_absoluteY="8dp" />
```

Activity.xml

```
<resources>
    <string name="app_name">TP2</string>
    <string-array name="day_array">
        <item>Lundi</item>
        <item>Mardi</item>
        <item>Mercredi</item>
        <item>Jeudi</item>
        <item>Vendredi</item>
        <item>Samedi</item>
        <item>Dimanche</item>
    </string-array>
</resources>
```

Strings.xml

Liste simple de chaines, chaque élément contient un jour de la semaine

- ListView
  - Une ***ListView*** avec des éléments simples (valeurs numériques, chaines) peut être construite de deux façons :
    - Dans la classe de l'activité, en créant et remplissant un adaptateur (de type « ***ArrayAdapter*** » par exemple) puis en liant cet adaptateur à la liste

Création d'une référence du composant XML « ListView »

```
ListView mylistView = findViewById(R.id.listView2);
String[] items = {"Lundi", "Mardi", "Mercredi", "Jeudi", "Vendredi", "Samedi", "Dimanche"};
ArrayAdapter<String> adapter = new ArrayAdapter<>( context: this, android.R.layout.simple_list_item_1, items);
mylistView.setAdapter(adapter);
```

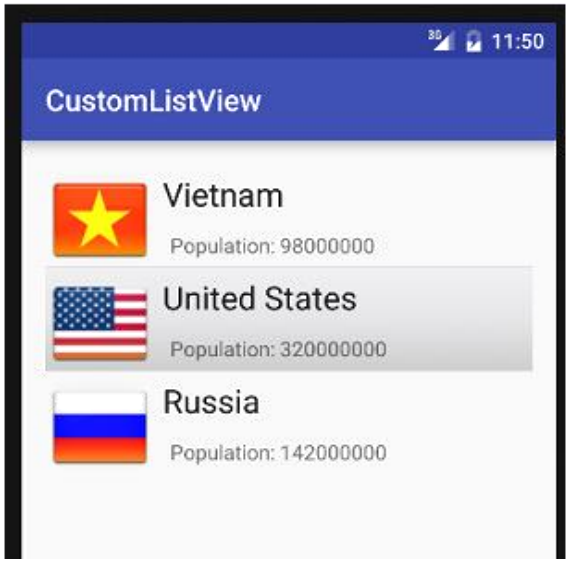
Création d'un adaptateur de type « ArrayAdapter »

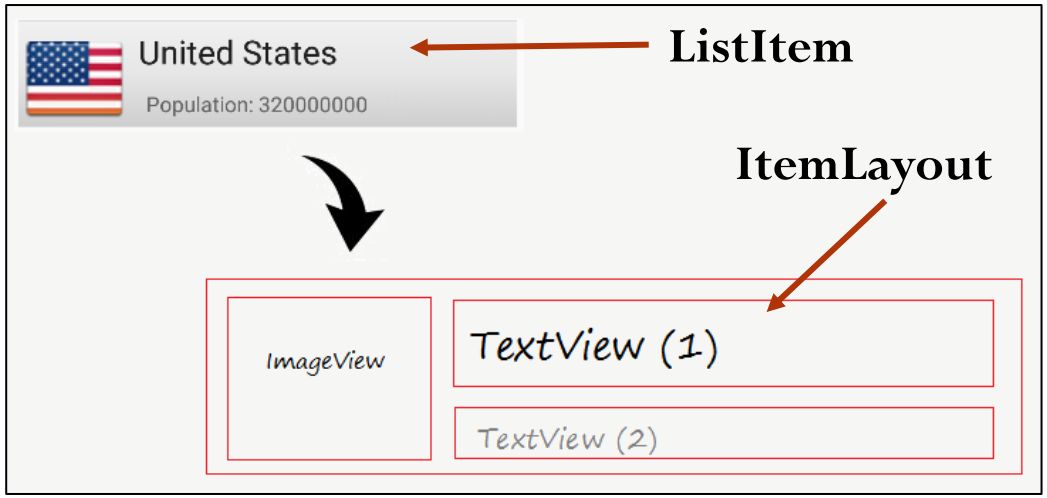
Layout des éléments de la liste

Association de l'adaptateur à la liste

Les éléments de la liste

- ## ListItem
  - Il existe des *layouts* prédéfinis pour les éléments « *ListItems* » tels que « *simple_list_item_1* », « *simple_list_item_2* », « *simple_spinner_item* » … (Ils sont utilisables avec un adaptateur de type *ArrayAdapter*)
  - On peut aussi créer des « *ItemLayout* » personnalisés

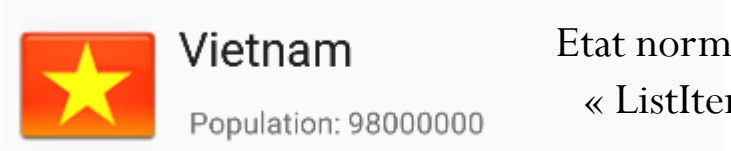




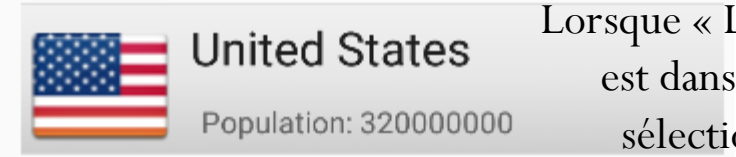
Il suffit de créer un fichier layout XML qui modélise la combinaison des composants , leur disposition et leurs propriétés

ListView Selector

- Pour rendre *ListView* plus beau, il vaut mieux personnaliser les effets, tels que modifier la couleur d'arrière-plan de *ListItem* lorsque le curseur se déplace dessus ou modifier la couleur d'arrière-plan lorsqu'il est sélectionné → C'est le rôle du *Selector*
- Un *Selector* est créé à travers un fichier XML dans le répertoire « *drawable* » des ressources



Etat normal du « ListItem »



Lorsque « ListItem » est dans l'état sélectionné

Lorsque « ListItem » est dans l'état cliqué

```
<?xml version="1.0" encoding="utf-8"?>
<selector xmlns:android="http://schemas.android.com/apk/
<!-- État cliqué -->
<item android:state_pressed="true"
    android:drawable="@color/colorPressed" />
<!-- État sélectionné -->
<item android:state_selected="true"
    android:drawable="@color/colorSelected" />
<!-- État au repos (par défaut) -->
<item android:drawable="@color/colorDefault" />
</selector>
```

- Enfin, il suffit d'associer le *selctor* à la liste comme suit :

```
<ListView
    android:id="@+id/listView2"
    android:listSelector="@drawable/list_selector"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
```

- GridView
  - C'est un composant qui fonctionne exactement comme une *ListView*. Il admet des éléments (*Items*) qui sont disposés en colonnes
  - Le nombre des colonnes est mentionné dans le code XML du composant *GridView*

GridView à deux colonnes

- GridView
  - Tout ce qui a été dit pour *ListView* reste valable pour *GridView*
  - Exemple de *GridView* simple avec *ArrayAdapter*

```
<GridView
    android:id="@+id/gridView"
    android:layout_width="344dp"
    android:layout_height="495dp"
    android:entries="@array/day_array"
    android:numColumns="2"
    tools:layout_editor_absoluteX="8dp"
    tools:layout_editor_absoluteY="8dp" />
```

Nombre des colonnes

Activity.xml

```
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_lv);
    String[] days = getResources().getStringArray(R.array.day_array);
    GridView s = (GridView) findViewById(R.id.gridView);
    ArrayAdapter<String> adapter = new ArrayAdapter<String>(this, android.R.layout.simple_list_item_1,days);
    s.setAdapter(adapter);
}
```

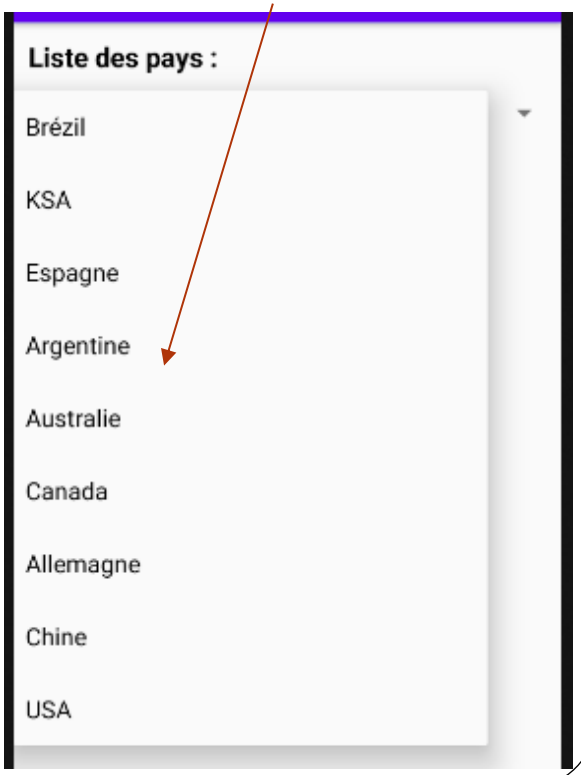
Activity.java

- Spinner

  - C'est un composant qui représente une liste déroulante
  - Il peut être créé et remplit au niveau des fichiers XML de ressources ou bien dans le code de l'activité à travers l'utilisation d'un adaptateur
  - Ici aussi on peut utiliser des gestionnaires prédéfinis et on peut créer des gestionnaires personnalisés en héritant la classe « *BaseAdapter* »

Elément sélectionné et affiché

Liste des éléments sélectionnables



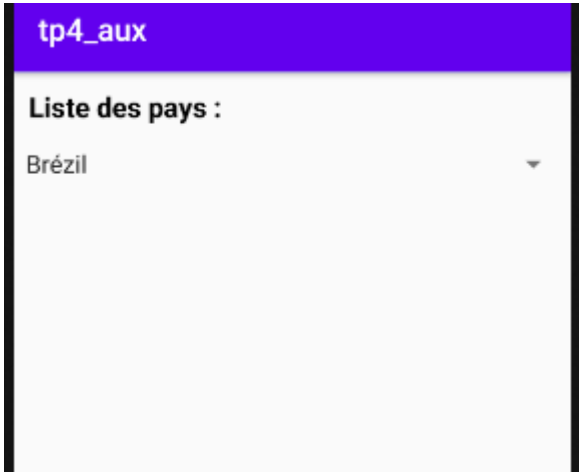
- Spinner
  - Exemple en XML uniquement

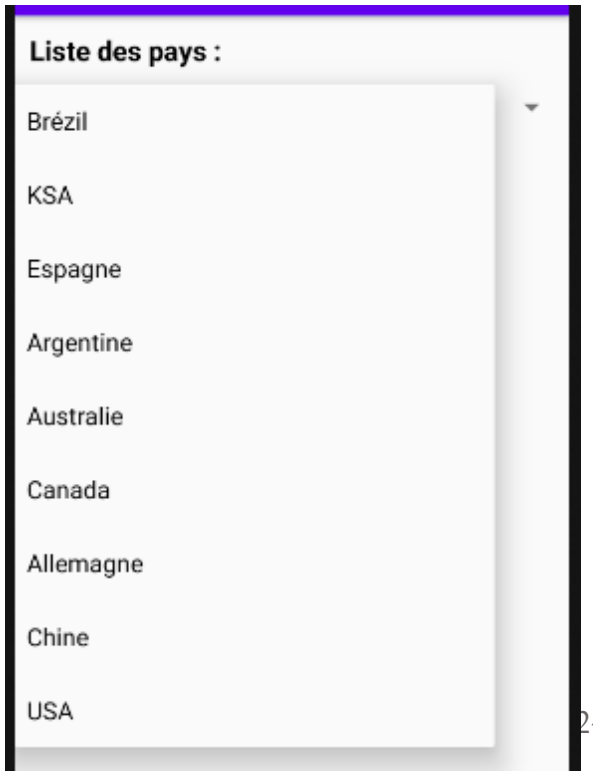
```xml
<Spinner
    android:id="@+id/spinner"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:entries="@array/NomsPays"
    android:visibility="visible" />
```

Activity.xml

```xml
<string-array name="NomsPays">
    <item>Brézil</item>
    <item>KSA</item>
    <item>Espagne</item>
    <item>Argentine</item>
    <item>Australie</item>
    <item>Canada</item>
    <item>Allemagne</item>
    <item>Chine</item>
    <item>USA</item>
</string-array>
```

strings.xml





- Spinner
  - Exemple de création dans le code de l'activité

```
<Spinner
    android:id="@+id/spinner"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:visibility="visible" />
```

Activity.xml

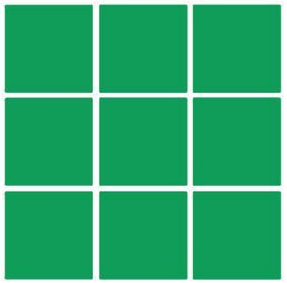
Layout prédéfinis de la liste des éléments sélectionnables

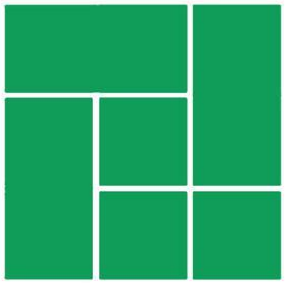
```
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_spinner);
    String []Pays = new String []{"Brézil", "KSA", "Espagne", "Agentine", "Australia",
    "Canada", "Allemagne", "Chine", "USA"};
    Spinner sp = findViewById(R.id.spinner);
    ArrayAdapter adapter = new ArrayAdapter( context: this, android.R.layout.simple_spinner_item,
            Pays);
    adapter.setDropDownViewResource(android.R.layout.simple_spinner_dropdown_item);
    sp.setAdapter(adapter);
}
```

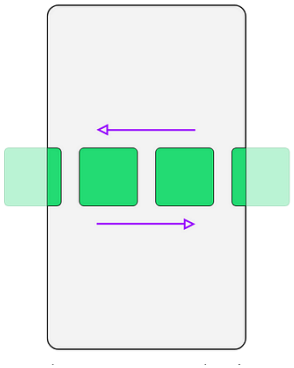
Activity.java

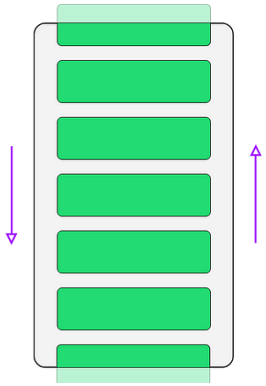
Layout prédéfinis de l'élément sélectionné et affiché

- RecyclerView
  - C'est un composant moderne, flexible et performant qui vient pour remplacer les anciens composants comme *ListView*, *GridView*, et *Gallery*
  - Il permet de mieux gérer de grandes quantités de données et offre une personnalisation plus fine
  - Le *RecyclerView* utilise un *LayoutManager* pour déterminer comment les éléments sont disposés (en liste verticale ou horizontale, grille à cellules de tailles homogènes ou hétérogènes). On trouve :
    - *LinearLayoutManager* : Affiche les éléments dans une liste verticale ou horizontale
    - *GridLayoutManager :* Affiche les éléments dans une grille
    - *StaggeredGridLayoutManager* :Dispose les éléments dans une grille décalée, où chaque élément peut avoir des dimensions différentes









- RecyclerView
  - Les éléments qui constituent une *RecyclerView* s'appellent *ItemView*
  - Le *RecyclerView* est associé à un *Adapter* qui permet de lier les données
  - L'adaptateur se charge de créer une vue pour chaque élément (*ItemView*) avec un *layout* (*ItemLayout*) à respecter
  - L'adaptateur se charge aussi du recyclage des éléments du *RecyclerView* (Au lieu de créer une nouvelle vue pour chaque élément visible, *RecyclerView* utilise des vues réutilisables, ce qui améliore les performances)
  - Le recyclage des éléments se base dur l'utilisation du *ViewHolder* qui organise et encapsule la logique d'affichage des éléments individuels et gère la gestion des interactions utilisateur et des événements directement au niveau des éléments
  - NB : Pour des interfaces modernes et performantes, *RecyclerView* est désormais le composant privilégié

MERCI DE VOTRE ATTENTION